

INFORME DE EXPERIMENTO

Comparación de Rendimiento: V1 Monolítica vs V2 Concurrente vs V3 Master-Worker

Sistema SITM-MIO — Cálculo de Velocidades Promedio por Ruta y Mes

Universidad Icesi — Ingeniería de Software IV

2026

1. Introducción

El Sistema Integrado de Transporte Masivo MIO de Cali opera aproximadamente 1.000 buses que emiten datagramas GPS cada 20–30 segundos, generando cerca de 2,5 millones de eventos diarios. El presente experimento evalúa tres estrategias de procesamiento para calcular la velocidad promedio por ruta activa y por mes, con el objetivo de determinar cuándo resulta rentable pasar de una solución monolítica a una solución concurrente o distribuida.

1.1 Descripción del problema

A partir de registros GPS crudos se calculan segmentos de trayecto consecutivos para cada vehículo (bus + ruta + viaje). Para cada segmento se calcula la distancia mediante la fórmula de Haversine y la velocidad instantánea. Los segmentos con gap mayor a 5 minutos, velocidad superior a 120 km/h o coordenadas inválidas son descartados. Finalmente se acumula distancia y tiempo por (lineId, año-mes) y se reporta la velocidad promedio.

1.2 Hipótesis

H1: La versión concurrente (V2) con múltiples workers será más rápida que la monolítica (V1) para volúmenes de datos grandes.

H2: La versión Master-Worker (V3) con el número óptimo de workers superará en rendimiento a V1 y V2.

H3: Los tres enfoques producirán resultados de velocidad idénticos (correctitud conservada).

2. Diseño Experimental

2.1 Variables independientes

Variable	Valores
Versión del sistema	V1 (monolítica), V2 (concurrente), V3 (master-worker)
Número de workers	1, 2, 4, 8 (solo V2 y V3)
Dataset	datagrams-MiniPilot.csv (fijo)

2.2 Variables dependientes (métricas)

Métrica	Descripción	Unidad
Tiempo de ejecución	Tiempo interno de Java (excluye startup JVM)	ms
Throughput	Datagramas procesados por segundo	filas/s
Uso de RAM	Memoria heap usada al finalizar	MB
CPU%	Porcentaje de CPU usado durante la ejecución	%
Speedup	$\text{tiempo_V1} / \text{tiempo_configuración}$	×
Eficiencia	$\text{speedup} / \text{número de workers}$	%
Correctitud	Diferencia en resultados vs V1 (diff)	líneas distintas

2.3 Metodología

Se realizaron **10 iteraciones por configuración**, para un total de 90 ejecuciones. Cada iteración procesa el mismo archivo de entrada (datagrams-MiniPilot.csv, 8.145.462 filas). Los resultados reportados corresponden a la media aritmética y la desviación estándar de las 10 iteraciones. Para el cálculo de CPU% se divide el tiempo interno de Java reportado por el tiempo de pared medido con `/usr/bin/time`.

2.4 Entorno de ejecución

Componente	Detalle
SO	Ubuntu 24.04 LTS (WSL2 sobre Windows 11)
JVM	OpenJDK 21.0.11 (HotSpot 64-bit Server VM)
Gradle	8.12
CPU	x86_64 (disponible para WSL)
Dataset	datagrams-MiniPilot.csv — 8.145.462 filas, ~169 MB comprimido
Rutas activas	111 rutas — lines-241-ActiveGT.csv

3. Descripción de las Versiones

3.1 V1 — Monolítica (baseline)

Procesa el archivo CSV línea a línea en un único hilo. Mantiene un mapa *previousByTrip* indexado por (busId, lineId, tripId) para rastrear el último punto válido de cada viaje. Es la solución de referencia (oracle) con la que se comparan V2 y V3 en términos de correctitud y rendimiento.

3.2 V2 — Concurrente (Thread Pool)

Un hilo reader lee y valida datagramas. Cada datagrama válido es despachado a uno de N workers mediante una *BlockingQueue* de capacidad 1.024, asignando por *lineId % N*. Esto garantiza que todos los datagramas de un mismo viaje lleguen al mismo worker en orden de archivo, preservando la semántica de V1. Los workers mantienen acumuladores locales; al terminar, el hilo principal fusiona los resultados parciales.

3.3 V3 — Master-Worker

El Master lee todos los datagramas válidos agrupándolos en memoria por lineId (preservando el orden de archivo). Una vez completada la lectura, encola un WorkPackage completo por lineId en una *LinkedBlockingQueue*. N workers consumen paquetes de la cola, procesan cada lineId de forma completamente independiente y devuelven acumuladores parciales. El Master fusiona todos los resultados y construye el reporte final. Este diseño elimina la sincronización por datagram de V2 a costa de mayor uso de memoria.

4. Resultados

4.1 Correctitud

Las tres versiones producen exactamente los mismos resultados de velocidad promedio por ruta y mes. La comparación con *diff* entre *monolith-results.csv*, *concurrent-results.csv* y *masterworker-results.csv* arroja **0 diferencias** en los tres pares posibles. La hipótesis H3 se confirma.

4.2 Tabla de resultados — Promedios y desviación estándar (10 iteraciones)

Versión	Workers	Avg(ms)	±Std	Min(ms)	Max(ms)	Tput/s	RAM(MB)	CPU%	Speedup	Efic.
monolith	1	17,129	±1,747	14,334	19,701	480,641	715	98.7%	1.00x	100%
concurrent	1	20,285	±1,032	18,945	22,152	402,563	590	99.1%	0.84x	84%
concurrent	2	35,373	±2,767	31,041	41,295	231,647	394	99.5%	0.48x	24%
concurrent	4	69,988	±9,440	54,493	84,822	118,526	291	99.8%	0.24x	6%
concurrent	8	109,973	±16,102	65,962	126,251	76,419	286	99.8%	0.16x	2%
master-worker	1	22,540	±3,471	16,006	27,902	370,531	3,084	98.4%	0.76x	76%
master-worker	2	21,222	±1,950	16,718	23,803	387,459	3,050	98.4%	0.81x	40%
master-worker	4	13,701	±421	12,874	14,560	595,066	2,992	98.5%	1.25x	31%
master-worker	8	18,278	±2,965	12,894	21,853	459,018	3,044	98.1%	0.94x	12%

Tabla 1. Promedios de 10 iteraciones. La fila resaltada en verde es la configuración más rápida.

4.3 Speedup y eficiencia

El speedup mide cuánto más rápida es una configuración respecto a V1 monolítica. La eficiencia mide qué fracción del speedup teórico máximo se alcanza con N workers (eficiencia = speedup / N × 100%).

Versión	Workers	Avg time (ms)	Speedup vs V1	Eficiencia
monolith	1	17,129	1.00x	100%
concurrent	1	20,285	0.84x	84%
concurrent	2	35,373	0.48x	24%
concurrent	4	69,988	0.24x	6%
concurrent	8	109,973	0.16x	2%
master-worker	1	22,540	0.76x	76%
master-worker	2	21,222	0.81x	40%
master-worker	4	13,701	1.25x	31%
master-worker	8	18,278	0.94x	12%

Tabla 2. Speedup y eficiencia de cada configuración.

5. Análisis de Resultados

5.1 Comportamiento de V2 — Concurrente

V2 presenta un comportamiento contraintuitivo: **el tiempo aumenta proporcionalmente con el número de workers**. Con 1 worker tarda 20,285 ms, con 2 workers 35,373 ms, con 4 workers 69,988 ms y con 8 workers 109,973 ms. Esto representa un **slowdown** de 5.4x al pasar de 1 a 8 workers.

La causa raíz es el diseño de la BlockingQueue con capacidad 1.024: el reader produce datagramas más rápido de lo que los workers los consumen cuando hay múltiples workers (mayor contención en la cola). Con más workers, la cola se llena frecuentemente y el reader se bloquea, generando un cuello de botella de sincronización que supera con creces el beneficio del procesamiento paralelo. Además, V2 es más lenta que V1 en todas las configuraciones: incluso con 1 worker, el overhead de la BlockingQueue (enqueue + dequeue por cada uno de los 8,1 millones de datagramas) es mayor que la ausencia total de sincronización en V1.

5.2 Comportamiento de V3 — Master-Worker

V3 exhibe el comportamiento esperado de una solución paralela: mejora con más workers hasta un punto óptimo, luego el overhead de coordinación vuelve a degradar el rendimiento. Con 1 worker: 22,540 ms, con 2 workers: 21,222 ms, con 4 workers: **13,701 ms (óptimo)**, con 8 workers: 18,278 ms.

La diferencia clave respecto a V2 es que V3 elimina la sincronización por datagrama: el Master agrupa todos los datagramas de cada lineld en memoria y despacha paquetes completos a los workers. Cada worker opera sin ninguna sincronización durante el procesamiento, lo que permite aprovechar el paralelismo real.

Con 4 workers, V3 logra un **speedup de 1.25x** respecto a V1, siendo la única configuración que supera a la solución monolítica.

5.3 Punto óptimo de distribución

Con el dataset MiniPilot (8,1 millones de filas, ~169 MB comprimido), el punto a partir del cual vale la pena distribuir es **V3 con 4 workers**. Las razones son:

- El workload es mayormente I/O-bound: la lectura del CSV es secuencial y limita el paralelismo.
- V2 nunca supera a V1 porque su diseño de despacho por datagrama introduce demasiado overhead.
- V3 con 4 workers es la única configuración con speedup > 1x (1.25x). Con 8 workers el overhead de gestionar más workers supera la ganancia por paralelismo.
- Para datasets mayores (como datagrams4Pilot.csv, 806 millones de filas), el beneficio de V3 sería más pronunciado ya que el tiempo de cómputo crecería más que el overhead.

5.4 Análisis de RAM

El consumo de RAM refleja las diferencias arquitectónicas: V1 usa 715 MB (solo acumuladores en memoria), V2 usa entre 285–627 MB dependiendo del número de workers (menos workers = más acumuladores por worker = más RAM), y V3 usa aproximadamente 2,992 MB en todas las configuraciones porque carga todos los datagramas válidos (7,093,594 filas) en memoria antes de

despacharlos.

5.5 Verificación de hipótesis

Hipótesis	Resultado	Evidencia
H1: V2 más rápida que V1	RECHAZADA	2 es más lenta en todas las configuraciones (speedup 0.16x–0.84x)
H2: V3 supera a V1 y V2 PARCIALMENTE CONFIRMADA	PARCIALMENTE CONFIRMADA	V3 con 4 workers (1.25x) supera a V1, y V3 siempre supera a V2
H3: Resultados idénticos	CONFIRMADA	diff entre los tres archivos de salida = 0 líneas distintas

Tabla 3. Verificación de hipótesis.

6. Conclusiones

La concurrencia no siempre mejora el rendimiento.

V2 demuestra que un diseño deficiente de la sincronización puede perjudicar seriamente el rendimiento. El despacho por datagrama a través de BlockingQueue genera contención creciente con el número de workers, resultando en un slowdown en todas las configuraciones.

V3 Master-Worker valida el patrón con el número correcto de workers.

El patrón Master-Worker es efectivo cuando el trabajo puede dividirse en unidades independientes (lineIds en este caso) y el overhead de coordinación es bajo respecto al trabajo por unidad. Con 4 workers se obtiene el mejor equilibrio.

El punto de quiebre depende del diseño, no solo del volumen.

Para este dataset, la distribución vale la pena únicamente con V3 y 4 workers. Con V2 nunca vale la pena independientemente del volumen, porque el diseño es fundamentalmente ineficiente. Para datasets mucho mayores, V3 con más workers podría escalar mejor dado que el trabajo por lineId crece proporcionalmente.

La correctitud se preserva en todos los escenarios.

Los tres enfoques producen resultados idénticos bit a bit, confirmando que la paralelización no introduce errores de cómputo cuando se respetan las dependencias de datos por viaje (busId + lineId + tripId).

El costo de V3 es la memoria.

V3 requiere aproximadamente 3 GB de RAM para cargar 7,1 millones de datagramas válidos en memoria. Para el dataset 4Pilot (806 millones de filas), esto escalaría a aproximadamente 25–30 GB, lo que podría hacer inviable V3 en servidores con poca RAM.

7. Datos Completos del Experimento

A continuación se presentan todas las 90 mediciones individuales del experimento, organizadas por versión y número de workers.

Versión	Workers	Iter	Time(ms)	Tput/s	RAM(MB)	CPU%
monolith	1	1	19,153	425,284	623.5	98.6%
monolith	1	2	19,701	413,454	646.4	99.1%
monolith	1	3	17,866	455,920	621.4	97.6%
monolith	1	4	18,909	430,772	629.7	98.4%
monolith	1	5	17,573	463,521	753.1	98.9%
monolith	1	6	16,108	505,678	757.0	98.9%
monolith	1	7	14,334	568,262	745.6	98.9%
monolith	1	8	16,307	499,507	754.1	98.8%
monolith	1	9	14,644	556,232	758.5	98.5%
monolith	1	10	16,699	487,781	862.7	98.9%
concurrent	1	1	20,352	400,229	617.1	99.1%
concurrent	1	2	22,152	367,708	624.5	98.9%
concurrent	1	3	22,038	369,610	631.8	98.8%
concurrent	1	4	20,412	399,053	516.6	99.2%
concurrent	1	5	18,976	429,251	508.3	99.1%
concurrent	1	6	20,045	406,359	626.5	99.1%
concurrent	1	7	19,771	411,990	506.8	99.1%
concurrent	1	8	20,362	400,033	623.3	99.1%
concurrent	1	9	18,945	429,953	617.8	99.1%
concurrent	1	10	19,797	411,449	627.4	99.1%
concurrent	2	1	41,295	197,251	348.6	99.6%
concurrent	2	2	35,289	230,822	351.4	99.4%
concurrent	2	3	34,726	234,564	421.3	99.3%
concurrent	2	4	33,735	241,454	351.8	99.4%
concurrent	2	5	36,469	223,353	427.7	99.6%
concurrent	2	6	33,802	240,976	421.7	99.5%
concurrent	2	7	32,607	249,807	418.5	99.6%
concurrent	2	8	37,056	219,815	352.0	99.5%

concurrent	2	9	37,707	216,020	424.3	99.5%
concurrent	2	10	31,041	262,410	423.4	99.3%
concurrent	4	1	54,493	149,477	346.4	99.7%
concurrent	4	2	62,680	129,953	285.6	99.7%
concurrent	4	3	68,276	119,302	280.5	99.7%
concurrent	4	4	61,601	132,229	288.3	99.8%
concurrent	4	5	67,200	121,212	281.9	99.7%
concurrent	4	6	64,877	125,552	283.4	99.8%
concurrent	4	7	73,772	110,414	290.9	99.8%
concurrent	4	8	84,822	96,030	282.0	99.8%
concurrent	4	9	78,828	103,332	284.6	99.8%
concurrent	4	10	83,326	97,754	283.2	99.8%
concurrent	8	1	110,339	73,822	291.1	99.8%
concurrent	8	2	114,419	71,190	279.7	99.8%
concurrent	8	3	114,801	70,953	282.2	99.8%
concurrent	8	4	114,357	71,228	285.8	99.8%
concurrent	8	5	112,950	72,116	288.0	99.8%
concurrent	8	6	119,101	68,391	286.0	99.8%
concurrent	8	7	121,609	66,981	284.4	99.8%
concurrent	8	8	126,251	64,518	284.1	99.8%
concurrent	8	9	99,940	81,504	289.4	99.8%
concurrent	8	10	65,962	123,487	288.6	99.8%
master-worker	1	1	16,006	508,901	2937.7	98.7%
master-worker	1	2	19,781	411,782	3167.3	98.9%
master-worker	1	3	20,135	404,542	3177.9	98.7%
master-worker	1	4	21,839	372,978	3025.2	97.9%
master-worker	1	5	22,584	360,674	3293.2	98.7%
master-worker	1	6	21,921	371,583	2990.4	98.8%
master-worker	1	7	22,000	370,248	2970.3	98.0%
master-worker	1	8	25,820	315,471	2823.1	97.3%
master-worker	1	9	27,407	297,204	3139.5	98.4%
master-worker	1	10	27,902	291,931	3312.7	98.5%

master-worker	2	1	22,699	358,847	2878.3	98.5%
master-worker	2	2	21,037	387,197	3022.9	98.7%
master-worker	2	3	21,878	372,313	3036.9	98.2%
master-worker	2	4	21,615	376,843	3070.3	98.7%
master-worker	2	5	22,589	360,594	3126.8	98.2%
master-worker	2	6	22,515	361,779	3019.1	98.5%
master-worker	2	7	23,803	342,203	3331.8	98.6%
master-worker	2	8	19,722	413,014	3112.3	97.9%
master-worker	2	9	19,648	414,570	3024.4	98.5%
master-worker	2	10	16,718	487,227	2875.9	98.6%
master-worker	4	1	12,874	632,706	2987.9	98.4%
master-worker	4	2	13,613	598,359	3067.5	98.4%
master-worker	4	3	13,897	586,131	2972.1	98.4%
master-worker	4	4	13,748	592,483	3056.2	98.5%
master-worker	4	5	13,658	596,388	2947.4	98.5%
master-worker	4	6	13,232	615,588	2981.4	98.5%
master-worker	4	7	13,969	583,110	2966.0	98.5%
master-worker	4	8	13,785	590,893	3067.3	98.6%
master-worker	4	9	14,560	559,441	2956.3	98.6%
master-worker	4	10	13,677	595,559	2915.4	98.5%
master-worker	8	1	12,894	631,725	2935.8	98.1%
master-worker	8	2	14,555	559,633	2929.4	98.3%
master-worker	8	3	15,491	525,819	2985.8	98.3%
master-worker	8	4	16,931	481,098	3150.1	98.0%
master-worker	8	5	19,188	424,508	3169.3	98.3%
master-worker	8	6	19,157	425,195	3179.8	97.8%
master-worker	8	7	21,853	372,739	3013.5	98.3%
master-worker	8	8	21,358	381,378	2982.4	98.4%
master-worker	8	9	20,926	389,251	3030.9	97.8%
master-worker	8	10	20,423	398,838	3058.5	98.1%

Tabla 4. Datos individuales de las 90 ejecuciones.